

MEMORIA TÉCNICA

Trabajo Grupal — 3ª Evaluación

Lenguajes de Marcas y Sistemas de Gestión de la Información

Gestión de datos de LaLiga con PostgreSQL + Aplicación Web JS

Autores:

Alex Stoica · Adrián Doñate · Hugo Álvarez

Índice

1. Introducción y descripción del proyecto	2
1.1 Objetivos	2
1.2 Criterios de evaluación cubiertos.....	2
2. Instalación y configuración de PostgreSQL.....	3
2.1 Versión utilizada	4
2.2 Alternativas a PostgreSQL para trabajar con JSON	4
3. Modelo Entidad-Relación	4
3.1 Diagrama E-R.....	5
3.2 Descripción de relaciones.....	5
3.3 Restricciones de integridad destacadas	6
4. Scripts SQL: DDL, DML y exportación JSON.....	6
4.1 Creación de tablas (DDL)	6
4.2 Inserción de datos (DML)	6
4.3 Consultas de exportación JSON.....	7
Consulta 2 — Agrupación (plantillas por equipo) en pgAdmin.....	7
Consulta 3 — Partidos con múltiples JOIN	8
Consulta 4 — Estadísticas de goleadores.....	8
Consulta 5 — Búsqueda y filtrado de delanteros	8
4.4 Ficheros JSON generados	9
5. Aplicación web (HTML + CSS + JS).....	9
5.1 Estructura de ficheros.....	9
5.2 Secciones y funcionalidades	9
5.3 Técnicas de manipulación del DOM	10
5.4 Recorrido de arrays y funciones de orden superior	10
6. Dificultades encontradas y soluciones	10
6.1 Error en la instalación de PostgreSQL.....	10
6.2 Formato incorrecto de los JSON exportados	11
7. Conclusiones	11

1. Introducción y descripción del proyecto

Este trabajo grupal de la 3.ª evaluación del módulo de Lenguajes de Marcas y Sistemas de Gestión de la Información (LMSGI) tiene como objetivo demostrar la capacidad de diseñar una base de datos relacional, exportar su información en formato JSON y consumir esos datos desde una aplicación web desarrollada únicamente con HTML, CSS y JavaScript puro.

El dominio elegido es la Liga Española de Fútbol Profesional (LaLiga), temporada 2024-2025, trabajando con datos reales de equipos, estadios, jugadores y partidos.

1.1 Objetivos

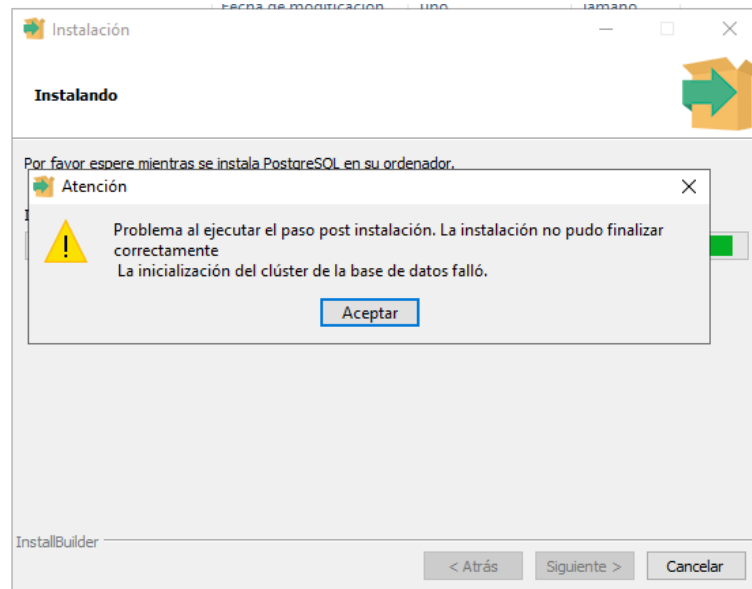
- Diseñar un modelo Entidad-Relación con al menos 5 tablas y relaciones correctamente tipadas.
- Implementar la base de datos en PostgreSQL con restricciones de integridad (NOT NULL, UNIQUE, CHECK, FK).
- Generar 5 ficheros JSON mediante consultas SQL con funciones nativas (json_agg, json_build_object, row_to_json).
- Desarrollar una aplicación web funcional que cargue esos datos y permita filtrar, ordenar y visualizar información.
- Documentar el proceso, las dificultades encontradas y las decisiones de diseño.

1.2 Criterios de evaluación cubiertos

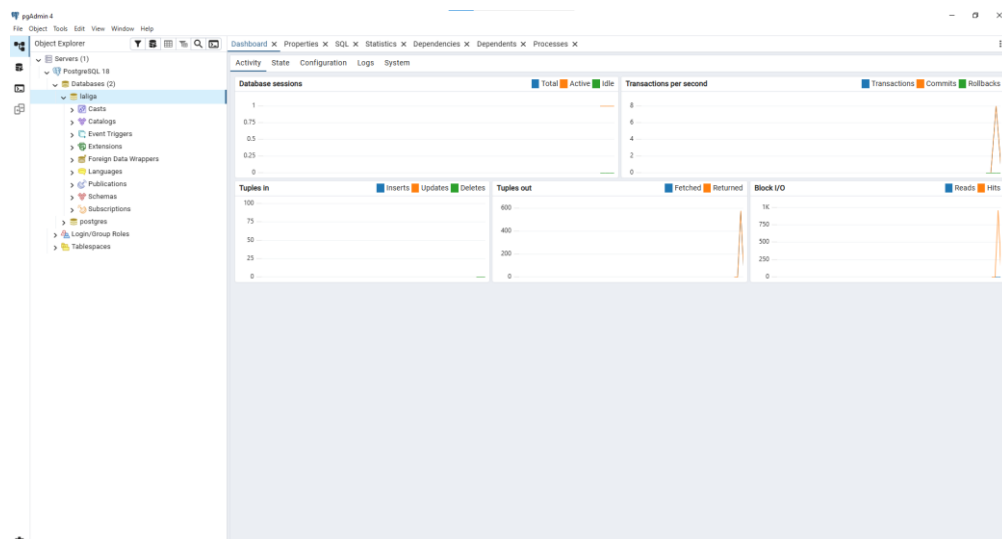
Crit.	Descripción	Fase
6a	Identifica métodos de almacenamiento en documentos de intercambio	Memoria
6b	Ventajas e inconvenientes del almacenamiento en JSON frente a otros formatos	Memoria
6c	Justifica la elección de PostgreSQL como tecnología de almacenamiento	PostgreSQL
6d	Identifica herramientas SQL y funciones JSON de PostgreSQL	PostgreSQL
6e	Usa consultas SQL para manipular y extraer datos como JSON	PostgreSQL
6f	Usa un SGBD relacional para almacenar y exportar información	PostgreSQL
6g	Genera documentos JSON a partir de una base de datos relacional	PostgreSQL
6h	Identifica características del sistema (VM, PostgreSQL, servidor HTTP)	Memoria
6i	Usa herramientas para gestionar la información (psql, pgAdmin, JS DOM)	Web

2. Instalación y configuración de PostgreSQL

PostgreSQL se instaló en el equipo local con Windows mediante el instalador oficial. Durante el proceso se encontró un error en la fase de post-instalación que impedía la inicialización del clúster de la base de datos.



El problema se resolvió ejecutando el instalador con permisos de Administrador y desactivando temporalmente el antivirus durante la instalación. Tras ello, el clúster se inicializó correctamente y el servicio PostgreSQL arrancó sin problemas.



2.1 Versión utilizada

- PostgreSQL 18 (instalador para Windows).
- pgAdmin 4 como herramienta gráfica de gestión y ejecución de consultas.
- VS Code con extensión SQLTools y Prettier para edición y formateo de scripts.

2.2 Alternativas a PostgreSQL para trabajar con JSON

Para los criterios 6a y 6b, se comparan las principales tecnologías de almacenamiento con soporte JSON:

Sistema	Ventajas para JSON	Inconvenientes
PostgreSQL	ACID, tipado fuerte, json_agg/json_build_object, SQL estándar	Configuración más compleja, curva de aprendizaje SQL
MongoDB	Nativo BSON/JSON, esquema flexible, escalado horizontal sencillo	Sin relaciones ACID entre colecciones, consultas menos expresivas
SQLite	Sin servidor, fichero único, fácil portabilidad, soporte JSON1	Sin concurrencia real, limitado a un solo escritor
MySQL / MariaDB	Muy extendido, soporte JSON desde v5.7/10.2	Funciones JSON menos potentes que PostgreSQL

Se eligió PostgreSQL porque combina integridad referencial estricta, SQL estándar y las funciones nativas más potentes para exportar datos relacionales directamente como JSON estructurado y anidado.

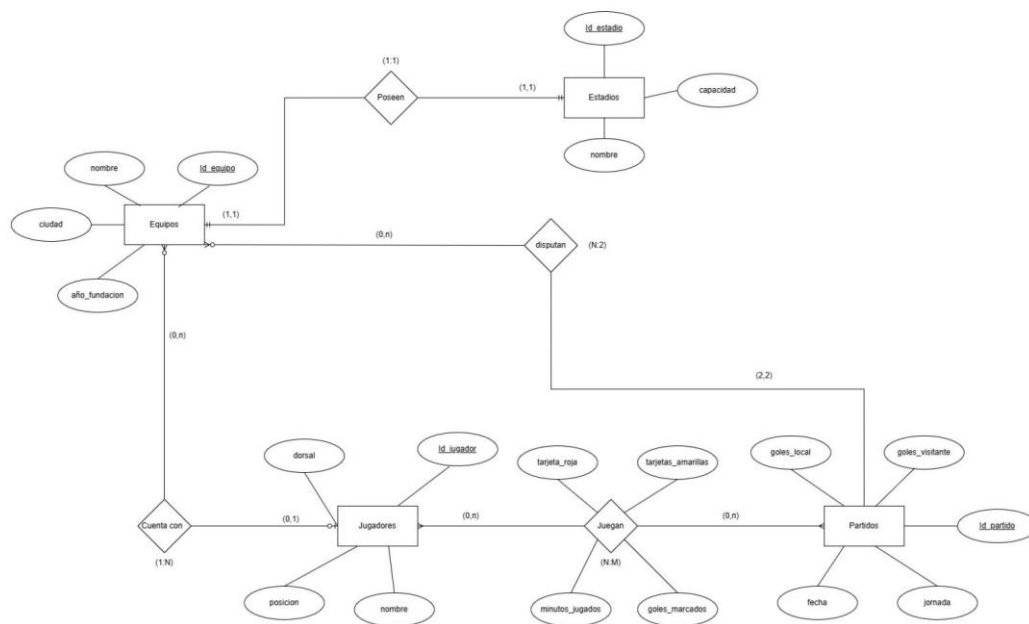
3. Modelo Entidad-Relación

El modelo fue diseñado con cinco entidades principales que cubren el dominio de LaLiga. A continuación se describe cada entidad y sus relaciones:

Tabla	Columnas principales	Descripción
equipos	id_equipo, nombre, ciudad, año_fundacion	Tabla principal. Clave primaria SERIAL.
estadios	id_estadio, nombre, capacidad, id_equipo	Relación 1:1 con equipos (FK con CASCADE).
jugadores	id_jugador, nombre, posicion, dorsal, id_equipo	Relación 1:N con equipos (FK SET NULL).
partidos	id_partido, jornada, fecha,	Relación N:2 con equipos. CHECK equipos distintos.

	id_equipo_local, id_equipo_visitante, goles_local, goles_visitante	
estadisticas_partido	id_partido, id_jugador, minutos_jugados, goles_marcados, tarjetas_amarillas, tarjeta_roja	Tabla intermedia N:M entre jugadores y partidos.

3.1 Diagrama E-R



3.2 Descripción de relaciones

- Equipos — Estadios: relación 1:1. Cada equipo posee exactamente un estadio. FK con ON DELETE CASCADE.
- Equipos — Jugadores: relación 1:N. Un equipo puede tener muchos jugadores. FK con ON DELETE SET NULL para preservar el historial.
- Equipos — Partidos: relación N:2. Cada partido involucra exactamente dos equipos (local y visitante). Dos FK distintas con CHECK que garantiza que no sean el mismo equipo.
- Jugadores — Partidos: relación N:M implementada con la tabla intermedia estadisticas_partido. Clave primaria compuesta (id_partido, id_jugador).

3.3 Restricciones de integridad destacadas

- CHECK en equipos: año_fundacion > 1850 AND año_fundacion <= año actual.

- CHECK en jugadores: dorsal entre 1 y 99; posición limitada a Portero, Defensa, Centrocampista o Delantero.
- CHECK en partidos: jornada entre 1 y 38; goles ≥ 0 ; equipos local y visitante distintos.
- CHECK en estadísticas_partido: minutos_jugados entre 0 y 120; tarjetas_amarillas entre 0 y 2.
- UNIQUE en equipos.nombre para evitar duplicados. NOT NULL en todos los campos obligatorios.

4. Scripts SQL: DDL, DML y exportación JSON

4.1 Creación de tablas (DDL)

Se crearon las cinco tablas en el siguiente orden para respetar las dependencias de claves ajenas:

```
CREATE TABLE equipos (
  id_equipo      SERIAL PRIMARY KEY,
  nombre         VARCHAR(100) NOT NULL UNIQUE,
  ciudad         VARCHAR(100) NOT NULL,
  año_fundacion  INTEGER CHECK (año_fundacion > 1850
    AND año_fundacion <= EXTRACT(YEAR FROM CURRENT_DATE))
);

CREATE TABLE estadisticas_partido (
  id_partido      INTEGER NOT NULL,
  id_jugador      INTEGER NOT NULL,
  minutos_jugados INTEGER DEFAULT 0 CHECK (minutos_jugados BETWEEN 0 AND 120),
  goles_marcados  INTEGER DEFAULT 0 CHECK (goles_marcados >= 0),
  tarjetas_amarillas INTEGER DEFAULT 0 CHECK (tarjetas_amarillas BETWEEN 0 AND 2),
  tarjeta_roja    BOOLEAN DEFAULT FALSE,
  PRIMARY KEY (id_partido, id_jugador),
  CONSTRAINT fk_estadistica_partido FOREIGN KEY (id_partido)
    REFERENCES partidos(id_partido) ON DELETE CASCADE,
  CONSTRAINT fk_estadistica_jugador FOREIGN KEY (id_jugador)
    REFERENCES jugadores(id_jugador) ON DELETE CASCADE
);
```

4.2 Inserción de datos (DML)

- 10 equipos con ciudad y año de fundación reales de LaLiga.
- 10 estadios con capacidades oficiales de la temporada 2024-25.
- 12 jugadores representativos con posición y dorsal correctos.
- 4 partidos en 2 jornadas con resultados coherentes (Real Madrid 4-0 Girona, Barça 1-0 Sevilla, Atleti 5-0 Villarreal, Valencia 2-2 Real Madrid).
- 4 registros de estadísticas: Bellingham 2 goles vs Girona, Vinicius 1 gol vs Girona, Lewandowski 1 gol vs Sevilla, Griezmann 2 goles vs Villarreal.

4.3 Consultas de exportación JSON

#	Tipo	Descripción
1	Listado general	JOIN entre equipos y estadios. Exporta id, nombre, ciudad, estadio y capacidad.
2	Agrupación por equipo	Subconsulta con json_agg anidado para obtener la plantilla completa de cada equipo.
3	Partidos con múltiples JOIN	Doble JOIN con la tabla equipos (local y visitante) ordenado por jornada y fecha.
4	Estadísticas de goleadores	JOIN con subquery que usa SUM + GROUP BY + HAVING para calcular goles totales.
5	Búsqueda y filtrado	Filtra jugadores por posición ('Delantero') con JOIN a equipos usando row_to_json.

Consulta 2 — Agrupación (plantillas por equipo) en pgAdmin

Object Explorer: Servers (1) > PostgreSQL 18 > Databases (2) > laliga

Query:

```
1 SELECT json_agg(
2     json_build_object(
3         'id_equipo', e.id_equipo,
4         'nombre_equipo', e.nombre,
5         'ciudad', e.ciudad,
6         'estadio', es.nombre,
7         'capacidad', es.capacidad
8     )
9 ) AS equipos_json
10 FROM equipos e
11 JOIN estadios es ON e.id_equipo = es.id_equipo;
```

Data Output: Showing rows: 1 to 1

equipos_json
[[{"id_equipo": 1, "nombre_equipo": "Real Madrid CF", "ciudad": "Madrid", "estadio": "Santiago Bernab\u00e9u", "capacidad": 81044}, {"id_equipo": 2, "nombre_equipo": "FC Barcelona", "ciudad": "...

Consulta 3 — Partidos con múltiples JOIN

Object Explorer: Servers (1) > PostgreSQL 18 > Databases (2) > laliga

Query:

```
1 SELECT json_agg(
2     json_build_object(
3         'equipo', e.nombre,
4         'fundacion', e.a\u00f1o_fundacion,
5         'jugadores', (
6             SELECT json_agg(
7                 json_build_object(
8                     'nombre', j.nombre,
9                     'posicion', j.posicion,
10                    'dorsal', j.dorsal
11                )
12            FROM jugadores j
13            WHERE j.id_equipo = e.id_equipo
14        )
15    )
16 ) AS plantillas_json
17 FROM equipos e;
```

Data Output: Showing rows: 1 to 1

plantillas_json
[[{"equipo": "Real Madrid CF", "fundacion": 1902, "jugadores": [{"nombre": "Vin\u00edcius J\u00fanior", "posicion": "Delfantero", "dorsal": 7}, {"nombre": "Jude Bellingham", "posicion": "Centroca...

Consulta 4 — Estadísticas de goleadores

The screenshot shows the PostgreSQL Query Editor interface. The left pane displays the database structure, including the 'laliga' database. The main pane shows a SQL query that uses a recursive CTE to generate a list of matches and then aggregates goal data into a JSON array.

```
1 SELECT json_agg(
2     json_build_object(
3         'jornada', p.jornada,
4         'fecha', p.fecha,
5         'equipo_local', el.nombre,
6         'goles_local', p.goles_local,
7         'goles_visitante', p.goles_visitante,
8         'equipo_visitante', ev.nombre
9     )
10 ) AS partidos_json
11 FROM partidos p
12 JOIN equipos el ON p.id_equipo_local = el.id_equipo
13 JOIN equipos ev ON p.id_equipo_visitante = ev.id_equipo
14 GROUP BY p.jornada, p.fecha
15 ORDER BY p.jornada, p.fecha;
```

The Data Output pane shows the results of the query, displaying a JSON array of match statistics.

partidos_json
[{"jornada": 1, "fecha": "2024-02-10", "equipo_local": "Real Madrid CF", "goles_local": 4, "equipo_visitante": "Girona FC"}]
[{"jornada": 1, "fecha": "2024-02-11", "equipo_local": "FC Barcelona", "goles_local": 1, "equipo_visitante": "Sevilla FC"}]
[{"jornada": 2, "fecha": "2024-02-17", "equipo_local": "Atlético de Madrid", "goles_local": 5, "equipo_visitante": "Villareal"}]
[{"jornada": 2, "fecha": "2024-02-18", "equipo_local": "Valencia CF", "goles_local": 2, "equipo_visitante": "Real Madrid C..."}]

Consulta 5 — Búsqueda y filtrado de delanteros

The screenshot shows the PostgreSQL Query Editor interface. The left pane displays the database structure, including the 'laliga' database. The main pane shows a SQL query that uses a recursive CTE to generate a list of players and then aggregates goal data into a JSON array.

```
1 SELECT json_agg(
2     json_build_object(
3         'jugador', j.nombre,
4         'equipo', e.nombre,
5         'goles_totales', stats.total_goles
6     )
7 ) AS goleadores_json
8 FROM jugadores j
9 JOIN equipos e ON j.id_equipo = e.id_equipo
10 JOIN (
11     SELECT id_jugador, SUM(goles_marcados) AS total_goles
12     FROM estadisticas_partido
13     GROUP BY id_jugador
14     HAVING SUM(goles_marcados) > 0
15 ) stats ON j.id_jugador = stats.id_jugador;
```

The Data Output pane shows the results of the query, displaying a JSON array of player statistics.

goleadores_json
[{"jugador": "Jude Bellingham", "equipo": "Real Madrid CF", "goles_totales": 2}, {"jugador": "Vinicius Júnior", "equipo": "Real Madrid CF", "goles_totales": 1}, {"jugador": "Robert Lewan..."}]

4.4 Ficheros JSON generados

The screenshot shows a file explorer window with the 'Descargas' (Downloads) folder selected. The left pane shows the navigation pane with 'Este equipo' (This PC) selected. The main pane shows a list of files with columns: Nombre, Fecha de modificación, Tipo, and Tamaño.

Nombre	Fecha de modificación	Tipo	Tamaño
1.Listado general.json	03/05/2026 18:47	Archivo JSON	2 KB
2.Agrupacion.json	03/05/2026 18:49	Archivo JSON	2 KB
3.Compleja multiple.json	03/05/2026 18:52	Archivo JSON	1 KB
4.Consulta estadísticas.json	03/05/2026 18:54	Archivo JSON	1 KB
5.Busqueda y filtrado.json	03/05/2026 18:55	Archivo JSON	1 KB

Los resultados se exportaron desde pgAdmin y se formatearon con VS Code (Prettier). El principal problema fue que PostgreSQL devuelve el JSON comprimido en una sola línea con comillas dobles escapadas ("""). El proceso de limpieza consistió en copiar el contenido de la celda, eliminar los escapes y aplicar Format Document para indentar correctamente.

5. Aplicación web (HTML + CSS + JS)

La aplicación web representa una interfaz de consulta de datos de LaLiga Temporada 24/25, construida únicamente con tecnologías estándar sin frameworks ni librerías externas.

5.1 Estructura de ficheros

- index.html — Estructura semántica con aside, main, section, article, footer.
- css/style.css — Layout flex, tarjetas con hover, panel modal, element <meter>, responsive.
- js/data.js — Datos JSON embebidos como constantes JS (datosEquipos, datosJugadores, datosPartidos, datosEstadisticas, datosAgrupacion).
- js/ui.js — Funciones de renderizado: Dom_Equipos, Dom_Jugadores, Dom_Partidos, Dom_Estadisticas, Dom_Panel_Detalles.
- js/events.js — Todos los addEventListener y lógica de filtrado/ordenación, envueltos en DOMContentLoaded.
- img/ — Imágenes de cabecera por sección y logo oficial LaLiga en SVG.

5.2 Secciones y funcionalidades

Sección	Funcionalidades	Técnicas JS usadas
Equipos	Filtro por nombre/ciudad, panel de detalles al clic	filter(), find(), addEventListener click/input
Jugadores	Filtro por nombre, select por equipo, ordenar por nombre/dorsal/equipo	filter(), sort(), localeCompare(), change/input events
Partidos	Filtro por jornada	filter() sobre array, evento change en select
Estadísticas	Gráfica de barras con <meter>	forEach(), cálculo de máximo, DOM dinámico

5.3 Técnicas de manipulación del DOM

- createElement / appendChild — creación dinámica de tarjetas <article> en cada sección.
- textContent — asignación de texto segura (sin riesgo de XSS).

- `classList.add` — aplicación de clases CSS a elementos generados dinámicamente.
- `style.display` — mostrar/ocultar secciones y el panel de detalles modal.
- `closest('.tarjeta-equipo')` — delegación de eventos para detectar clics en tarjetas.

5.4 Recorrido de arrays y funciones de orden superior

- `forEach` — renderizar cada elemento de los arrays de datos.
- `filter` — filtros por nombre, ciudad, equipo y jornada en tiempo real.
- `find` — buscar el equipo en `datosAgrupacion` al clicar una tarjeta.
- `sort + localeCompare` — ordenar jugadores por nombre o equipo alfabéticamente.
- `sort` con comparador numérico — ordenar por dorsal de menor a mayor.
- Spread operator `[...array]` — no mutar el array original al ordenar.

6. Dificultades encontradas y soluciones

6.1 Error en la instalación de PostgreSQL

Al instalar PostgreSQL en Windows apareció el mensaje "La inicialización del clúster de la base de datos falló", impidiendo que el servicio arrancara correctamente (ver Fig. 1).

Solución: Ejecutar el instalador como Administrador y deshabilitar temporalmente el antivirus durante la instalación.

6.2 Formato incorrecto de los JSON exportados

Al exportar desde pgAdmin, los JSON aparecían en una sola línea con comillas dobles escapadas (`\"\"`), dificultando su uso directo en la aplicación.

Solución: Copiar el contenido de la celda, limpiar manualmente los escapes y formatear con VS Code (Prettier). Se procesaron los 5 ficheros JSON de la misma manera.

7. Conclusiones

Este ha sido nuestro primer proyecto combinando base de datos, JSON y una página web completa con HTML, CSS y JavaScript. Ha sido un buen reto porque había que hacer funcionar las tres partes juntas y que se entendiesen entre sí.

Lo más complicado fue la parte de los eventos en JavaScript, entender cómo conectar cada interacción del usuario con la lógica de filtrado y renderizado del DOM.

También nos ha quedado mucho más claro para qué sirve realmente JSON: no es un lenguaje de programación sino un formato para transportar datos entre sistemas, en nuestro caso entre la base de datos y la página web.

En general estamos contentos con el resultado y hemos aprendido bastante, sobre todo con la manipulación del DOM y las consultas SQL con JSON.

